checkout step. Users will click on the advertisement or not, resulting in reward signals that get relayed back to the agent for learning.

Figure 2.6. A computational graph for a simple 10-armed contextual bandit. The get_state() function returns a state value, which is transformed into a one-hot vector that becomes the input data for a two-layer neural network. The output of the neural network is the predicted reward for each possible action, which is a dense vector that is run through a softmax to sample an action from the resulting probability distribution over the actions. The chosen action will return a reward and updates the state of the environment. θ1 and θ2 represent the weight parameters for each layer. The , , and symbols denote the natural numbers (0, 1, 2, 3, ...), the real numbers (a floating-point number, for our purposes), and a probability, respectively. The superscript indicates the length of the vector, so 10 represents a 10-element vector where each element is a probability (such that all the elements sum to 1).

Figure 2.7. A training graph showing the average rewards for playing the contextual bandit simulator using a two-layer neural network as the action-value function. We can see the average reward rapidly increases during training time, demonstrating our neural network is successfully learning.

Figure 2.8. A simplified MDP diagram with three states and two actions. Here we model the parenting decision process for taking care of an infant. If the baby is crying, we can either administer food or not, and with some probability the baby will transition into a new state, and we'll receive a reward of -1, +1, or +2 (based on the baby's satisfaction).

Figure 2.9. The general process of a reinforcement learning algorithm. The environment produces states and rewards. The agent takes an action, at, given a state, st, at time t and receives a reward, rt. The agent's goal is to maximize rewards by learning to take the best actions in a given state.

Chapter 3. Predicting the best states and actions: Deep Q-networks

Figure 3.1. This is a simple Gridworld game setup. The agent (A) must navigate along the shortest path to the goal tile (+) and avoid falling